

Manual do Usuário

Risco de Fogo Previsto em Python — INPE_FireRiskModel v2.2

Versão 1.4 · 5 de agosto de 2026

Substitui: rf_previsto_1-5dias_2023.sh e rf_previsto_1-2_semanas_2024.sh (bash + NCL)

Programa Queimadas — <http://www.inpe.br/queimadas/>

Sumário

1. Visão geral
2. Requisitos
3. Instalação
4. Uso
5. Script genérico para qualquer horizonte (rf_previsto.py)
6. Preparo dos dados de entrada do GFS (prepara_gfs.py)
7. Saídas
8. Logs e monitoramento
9. Solução de problemas
10. Testes de validação
11. Segurança
12. Suporte

1. Visão geral

Este pacote calcula o Risco de Fogo (RF) previsto em resolução de 1 km para a América do Sul, combinando a precipitação observada do IMERG (últimos 119 dias), as previsões do GFS (precipitação, temperatura e umidade relativa a 2 m), o mapa de vegetação (MapBiomias/IGBP) e a topografia (GTOPO30). São três programas:

- **rf_previsto_1_5dias.py** — gera 19 previsões, de +6 h até +4 dias 18 UTC, a cada 6 horas (produto RF_PREV).
- **rf_previsto_1_2_semanas.py** — gera 2 previsões, para +7 e +14 dias às 18 UTC (produto RF_PREV_SEMANAL).
- **rf_previsto.py** — script genérico: gera o RF para qualquer horizonte de previsão e diferentes fontes de dados (GFS, Eta, BESM) informados na linha de comando (seção 5).

Ambos usam o módulo comum `rf_core.py` e produzem, para cada horário de previsão, um NetCDF (RF.PREV.YYYYMMDDHH.nc) e um GeoTIFF (RF.PREV.YYYYMMDDHH.tif), além dos produtos derivados (links D1–D5, cópias T0–T4/T7/T14 e fogograma).

2. Requisitos

2.1 Software

Python 3.9 ou superior e as bibliotecas:

```
pip install numpy xarray netCDF4 rasterio pyyaml
```

Para rodar o teste de validação, instale também o `scipy`:

```
pip install scipy
```

Não são mais necessários: NCL, cdo, gdal (binário), GNU parallel nem o ambiente `conda ncl_stable`.

2.2 Dados de entrada

Dado	Local esperado	Observação
Precipitação IMERG (diária)	data/output/2.2/Precipitation-2_2/AAAA/MM/INPE_FireRiskModel_2.2_Precipitation_AAAAMMD D.nc	119 dias anteriores à execução; variável <code>prec</code>
Previsões do GFS	data/output/2.2/GFS/netcdf/AAAAMMD00/	GFS.PREV.PREC.* e GFS.PREV.TEMP2m.RH2m.*
Mapa de vegetação 1 km	data/input/Veg_Map_2020/Merge_MapBiomias_V5_IGBP_C6_<ano>.nc	Band1, classes 0–7, sul→norte; anos ≥ 2020 usam 2019
Topografia 1 km	data/input/topografia/GeoTOPOAmericaSulCentral_V3.nc	Band1 (metros), mesma grade da vegetação

Todos os caminhos partem de `/home/queimadas/INPE_FireRiskModel` e estão definidos como constantes no topo de cada script (seção "Configurações"). Para rodar em outra máquina, basta editar a constante `BASE`.

3. Instalação

Copie os arquivos para o diretório de scripts do modelo (os testes são opcionais):

```
rf_core.py
rf_fontes.py
rf_config.py
config_exemplo.yaml
rf_previsto_1_5dias.py
rf_previsto_1_2_semanas.py
rf_previsto.py
prepara_gfs.py
teste_rf.py
teste_rf_previsto.py
teste_rf_multifonte.py
teste_prepara_gfs.py
```

Os scripts devem ficar no mesmo diretório (os orquestradores fazem import rf_core). Se desejar, torne-os executáveis:

```
chmod +x rf_previsto_1_5dias.py rf_previsto_1_2_semanas.py
```

4. Uso

4.1 Execução operacional (data de hoje)

```
python3 rf_previsto_1_5dias.py
python3 rf_previsto_1_2_semanas.py
```

O comportamento é o mesmo dos shell scripts originais: as datas são calculadas a partir do dia corrente, as saídas vão para os mesmos diretórios e, ao final, os arquivos são publicados nos servidores.

4.2 Opções de linha de comando

Opção	Efeito	Padrão
--data-final YYYYMMDD	Executa como se "hoje" fosse a data informada (reprocessamento)	data corrente
--jobs N	Número de previsões calculadas em paralelo	20 (diário) / 2 (semanal)
--sem-envio	Não publica nos servidores (geoserver/lftp/scp) — útil para testes	envio ativado

Exemplos:

```
# Reprocessar 1º/08/2026 sem publicar, usando 8 processos
python3 rf_previsto_1_5dias.py --data-final 20260801 --jobs 8 --sem-envio

# Rodada semanal de teste
python3 rf_previsto_1_2_semanas.py --sem-envio
```

4.3 Agendamento (cron)

Substitua as entradas existentes que chamavam os shell scripts, por exemplo:

```
30 6 * * * cd /home/queimadas/INPE_FireRiskModel/scr/risco_fogo/RF_Previsto \
&& /usr/bin/python3 rf_previsto_1_5dias.py \
```

```
>> /home/queimadas/INPE_FireRiskModel/log/cron.rf_prev.log 2>&1
```

5. Script genérico para qualquer horizonte (rf_previsto.py)

O `rf_previsto.py` generaliza os dois produtos: os horizontes de previsão são informados na linha de comando, contados a partir das 00 UTC da data do modelo. Um horizonte pode ser escrito como Nh (horas), Nd (dias), Nm (meses de calendário), combinações como 4d18h ou 1m15d, ou como data/hora absoluta YYYYMMDDHH.

5.1 Exemplos

```
# Um único horizonte: 36 horas
python3 rf_previsto.py --horizontes 36h

# Lista de horizontes: 1, 3, 7 e 14 dias às 18 UTC
python3 rf_previsto.py --horizontes 18h,2d18h,6d18h,13d18h

# Intervalo (equivalente ao produto diário de 1 a 5 dias)
python3 rf_previsto.py --de 6h --ate 4d18h --passo 6h

# Equivalente ao produto semanal (7 e 14 dias)
python3 rf_previsto.py --horizontes 7d18h,14d18h --rb-max 0.8 --fallback-gfs

# Data/hora absoluta com reprocessamento
python3 rf_previsto.py --data-final 20260801 --horizontes 2026080618

# Eta: previsões mensais de 1 a 13 meses
python3 rf_previsto.py --fonte eta --de 1m --ate 13m --passo 1m

# BESM (acúmulos de 12 h agregados automaticamente): 6 meses
python3 rf_previsto.py --fonte besm --horizontes 6m
```

5.2 Opções

Opção	Efeito	Padrão
--horizontes LISTA	Horizontes separados por vírgula (Nh, Nd, Nm, combinações ou YYYYMMDDHH)	—
--fonte NOME	Fonte de previsão: gfs, eta, besm ou outra definida via JSON (seção 5.4)	composição legada (GFS)
--config-fontes ARQ	Arquivo JSON que ajusta/acrescenta fontes (nomes de arquivos, variáveis, frequência)	—
--de / --ate / --passo	Intervalo de horizontes relativo (pode ser combinado com --horizontes)	passo 6h
--rb-max X	Risco básico máximo (o produto semanal usa 0.8)	0.9
--produto NOME	Subdiretório de saída em data/output/2.2/	RF_PREV_CUSTOM
--base DIR	Diretório base do modelo (permite rodar fora da produção)	/home/queimadas/INPE_FireRiskModel
--fallback-gfs	Se o GFS do horário exato não existir, usa o horário anterior do mesmo dia (generaliza a cópia 12 UTC → 18 UTC do produto semanal)	desativado
--sem-tif	Gera apenas os NetCDF, sem GeoTIFF	TIF ativado
--fogograma	Gera também um único NetCDF com todos os horizontes	desativado

Opção	Efeito	Padrão
--data-final / --jobs N	Como nos demais scripts	hoje / 4

As saídas seguem o mesmo formato dos demais produtos (RF.PREV.YYYYMMDDHH.nc e .tif), gravadas em data/output/2.2/<produto>/netcdf|tif/<modelo>. O script genérico não gera links D1–D5, cópias T7/T14 nem faz envio a servidores — para os produtos operacionais, use os scripts dedicados.

5.3 Fontes de dados e horizontes longos (--fonte)

O alcance depende da fonte de previsão escolhida com --fonte:

Fonte	Alcance	Frequência padrão dos acúmulos
gfs	~16 dias	1 dia
eta	até 13 meses	1 dia
besm	até 13 meses	1 dia (arquivo único por variável)

Sem --fonte, o script usa a composição original dos produtos operacionais: 119 dias de IMERG observado + o acumulado do GFS do dia previsto (idêntica aos scripts dedicados, limitada ao alcance do GFS).

Com --fonte, a série diária de 120 tempos que antecede cada data prevista é montada de forma completa: IMERG observado para os dias anteriores à rodada e precipitação PREVISTA da fonte para os dias entre a rodada e a data válida. É isso que viabiliza horizontes de semanas a 13 meses (Eta/BESM) — num horizonte de 13 meses, os 120 dias da janela são inteiramente previstos. Os acúmulos da fonte podem vir em qualquer frequência (1h, 12h, 1d): são somados para o passo diário automaticamente, e campos em grade diferente da do IMERG são regradeados por interpolação bilinear. O horizonte pedido é validado contra o alcance da fonte (horizonte_max_dias).

5.4 Configuração das fontes (rf_fontes.py e --config-fontes)

Cada fonte é descrita em rf_fontes.py por: **layout** dos arquivos (por_tempo = um arquivo por horário previsto, como o GFS deste pipeline; serie = um arquivo por variável com todos os tempos da rodada, como o BESM T062), subdiretório dos dados, padrões de nome dos arquivos (com os marcadores {modelo} e {valida}), nomes das variáveis (var_prec, var_temp, var_ur), frequência dos acúmulos (freq_prec: 1h, 12h, 1d), tipo de acumulação (intervalo = acumulado do próprio intervalo; desde_inicio = acumulado desde o início da rodada), unidades (unidade_temp: K/C; unidade_ur: %/frac) e alcance (horizonte_max_dias).

A fonte besm já vem configurada com a convenção real do BESM T062 (pacote besm_queimada do CPTEC): layout serie com tmp_prec.nc (mm/dia), tmp_t2mt.nc (K, média diária) e tmp_rsmt.nc (% 0–100) em BESM/netcdf/<modelo>/, 396 tempos diários (rodada+1 até rodada+13 meses), grade gaussiana ~1,875° com latitude norte→sul (invertida automaticamente na leitura). Como a série começa em rodada+1, o dia da própria rodada é preenchido com o IMERG observado, se existir, ou aproximado pelo primeiro dia da série (aviso no log). Os padrões do Eta seguem PROVISÓRIOS — ajuste-os à convenção real com um JSON, sem alterar o código:

```
{
  "eta": {
    "subdir": "ETA/netcdf/{modelo}",
    "padrao_prec": "ETA.PREV.PREC.{modelo}.{valida}.nc",
```

```

    "padrao_temp_ur": "ETA.PREV.TEMP2m.RH2m.{modelo}.{valida}.nc",
    "var_prec": "prec",
    "freq_prec": "1d",
    "horizonte_max_dias": 396
  },
  "besm": { "freq_prec": "12h" }
}

python3 rf_previsto.py --fonte eta --horizontes 13m --config-fontes
fontes.json

```

Os arquivos das fontes devem ser NetCDF em grade regular lat/lon (como os do GFS já pré-processados neste pipeline); saídas nativas (grib do Eta, espectral do BESM) precisam ser convertidas antes. Novas fontes podem ser acrescentadas no mesmo JSON — o nome da chave passa a valer em --fonte.

5.5 Arquivo de configuração YAML (--config)

O --config arquivo.yaml centraliza toda a configuração dos dados de entrada num único arquivo YAML (ou JSON), com seções opcionais — o que não for informado usa os padrões da produção, e a linha de comando sempre prevalece sobre o arquivo:

Seção	Conteúdo
base	Diretório base do modelo; os caminhos relativos são ancorados nele
caminhos	Onde estão os dados de entrada: imerg_dir, imerg_subpastas ({ano}/{mes}), imerg_padrao ({data}), mapa_vegetacao ({ano_veg}), topografia, log
fontes	Mesmas chaves do --config-fontes: ajusta ou acrescenta fontes (gfs, eta, besm, ...)
execucao	Padrões da linha de comando: fonte, horizontes ou de/ate/passo, data_final, rb_max, produto, jobs, fallback_gfs, sem_tif, fogograma

```

base: /home/queimadas/INPE_FireRiskModel
caminhos:
  imerg_dir: data/output/2.2/Precipitation-2_2
  imerg_padrao: "INPE_FireRiskModel_2.2_Precipitation_{data}.nc"
fontes:
  besm: { horizonte_max_dias: 396 }
execucao:
  fonte: besm
  de: 1m
  ate: 13m
  passo: 1m
  produto: RF_PREV_BESM

python3 rf_previsto.py --config config.yaml # tudo do arquivo
python3 rf_previsto.py --config config.yaml --horizontes 6m # CLI prevalece

```

O arquivo config_exemplo.yaml traz o modelo completo comentado; as seções são validadas na leitura (chave desconhecida gera erro com a lista das válidas). Requer PyYAML (pip install pyyaml) — arquivos .json funcionam sem ele.

6. Preparo dos dados de entrada do GFS (prepara_gfs.py)

O prepara_gfs.py baixa a previsão do GFS 0,25° da NOAA e grava os arquivos GFS.PREV.PREC.* (prec em mm/dia) e GFS.PREV.TEMP2m.RH2m.* (T2m em K, UR2m em %) na convenção da seção 2.2, prontos para o rf_previsto.py. Requer o pygrib (python3 -m pip install pygrib).

Importante: o serviço OpenDAP do NOMADS foi aposentado pela NOAA (Service Change Notice 25-81, efetivo em 23/02/2026). O script usa os serviços vigentes indicados no próprio aviso:

Método	Descrição
s3 (padrão)	"Fast download method": lê o índice .idx de cada horário e baixa por byte-range só as 3 mensagens GRIB necessárias, no espelho oficial da AWS (noaa-gfs-bdp-pds), ~2 MB por horário
nomads	O mesmo fast download, direto no HTTPS do NOMADS (/pub/data/nccf/com/gfs/prod/...)
filtro	Grib filter do NOMADS (recorte de variáveis/região no servidor; URL ajustável via --url-filtro)

```
python3 prepara_gfs.py # rodada de hoje 00 UTC, 16 dias
python3 prepara_gfs.py --data 20260805 --config config.yaml
python3 prepara_gfs.py --simular # só mostra o plano e a URL
python3 prepara_gfs.py --metodo nomads # se a AWS estiver bloqueada
```

Opções principais: --data/--rodada (rodada), --dias (alcance, padrão 16), --passo (validades, padrão 6 h), --dominio latS,latN,lonW,lonE, --acumulo 24h|dia, --jobs (downloads simultâneos), --base/--config (destino), --sobrescrever e --simular.

Sobre a precipitação: o APCP do GFS vem em "baldes" (6 h até +240 h; 12 h de +240 h a +384 h). O acumulado diário de cada validade soma os baldes que cobrem as 24 h anteriores, com o intervalo lido do próprio GRIB — a transição 6 h/12 h é automática, e validades sem arquivo além de +240 h (fora do passo de 12 h) são puladas com aviso. Fluxo típico de uma rodada:

```
python3 prepara_gfs.py --config config.yaml # 1. baixa e prepara o GFS
python3 rf_previsto.py --de 6h --ate 4d18h --passo 6h # 2. calcula o RF
```

7. Saídas

7.1 Produto diário (1 a 5 dias)

Saída	Local
NetCDF por horário (19 arquivos)	data/output/2.2/RF_PREV/netcdf/<modelo>/RF.PREV.YYYYMMDDHH.nc
GeoTIFF por horário	data/output/2.2/RF_PREV/tif/<modelo>/RF.PREV.YYYYMMDDHH.tif
Links para o mapserver (D1–D5, 18 UTC)	dados/mapfiles/tmp/RF.PREV.D<dia>.tif
Cópias T0–T4 (18 UTC)	.../tif/<modelo>/RF.PREV.T<d>.YYYYMMDD18.tif
Fogograma (todos os horários juntos)	data/output/2.2/fogograma/RF.PREV.<data>00.nc

7.2 Produto semanal (1 a 2 semanas)

Saída	Local
NetCDF (2 arquivos: +7 e +14 dias)	data/output/2.2/RF_PREV_SEMANAL/netcdf/<modelo>/
GeoTIFF T7 e T14	.../RF_PREV_SEMANAL/tif/<modelo>/RF.PREV.T7.tif e

Saída	Local
	RF.PREV.T14.tif

7.3 Formato dos arquivos

O NetCDF contém a variável `rbf(time, lat, lon)` com o RF em $[0, 1]$, arredondado a 2 casas decimais, valor ausente `-999` e eixo de tempo na data/hora da previsão. O GeoTIFF está em EPSG:4326, orientado de norte para sul, com nodata `-999`, compressão LZW e organização em tiles.

Interpretação usual do RF: mínimo (< 0.15), baixo ($0.15-0.40$), médio ($0.40-0.70$), alto ($0.70-0.95$) e crítico (≥ 0.95).

8. Logs e monitoramento

Log	Conteúdo
<code>log/log.<modelo>.<previsão></code>	Progresso e eventuais erros de cada previsão (um por horário)
<code>log.falta.arquivos.prev.prec.txt</code> (diretório de execução)	Arquivos IMERG esperados e não encontrados
Saída padrão	Datas da rodada, horários processados e tempo total

O código de saída é 0 em caso de sucesso e 1 se alguma previsão não foi gerada (mensagem "PROBLEMA - FALTAM ARQUIVOS EM <dir>"), o que permite monitorar a rodada no cron da mesma forma que antes.

9. Solução de problemas

Sintoma	Causa provável	O que fazer
Número de tempos de precipitação insuficiente: N (esperado 120)	Faltam arquivos IMERG no período de 119 dias	Verifique <code>log.falta.arquivos.prev.prec.txt</code> e reponha os arquivos
PROBLEMA - FALTAM ARQUIVOS EM ...	Uma ou mais previsões falharam	Consulte <code>log/log.<modelo>.<previsão></code> do horário faltante
<code>FileNotFoundError</code> em arquivo <code>GFS.PREV.*</code>	A rodada do GFS ainda não foi processada para o dia	Aguarde/reprocesse a etapa do GFS e rode novamente
A topografia (...) não tem a mesma grade do mapa de vegetação (...)	Arquivo de topografia ou vegetação trocado	Confira os arquivos em <code>data/input/</code>
Consumo excessivo de memória / máquina lenta	Muitos processos simultâneos em grade de 1 km	Reduza <code>--jobs</code>
Falha no envio (lftp/scp)	Rede ou credenciais	O cálculo não é afetado; reenvie manualmente ou rode novamente
<code>ModuleNotFoundError</code> : rasterio (ou outra biblioteca)	Dependências não instaladas no Python usado	<code>pip install numpy xarray netCDF4 rasterio</code>

10. Testes de validação

Após instalar ou alterar qualquer coisa, rode:

```
python3 teste_rf.py          # valida o núcleo do cálculo (rf_core)
python3 teste_rf_previsto.py # valida o script genérico de ponta a ponta
python3 teste_rf_multifonte.py # valida o modo multifonte (Eta 13m, BESM 12h,
JSON)
python3 teste_prepara_gfs.py # valida o preparo do GFS (idx, baldes, NetCDF)
```

O primeiro teste cria dados sintéticos em /tmp/teste_rf, executa o cálculo completo e compara com uma implementação de referência fiel ao NCL; a saída esperada termina com "TODOS OS TESTES PASSARAM". O segundo monta uma árvore com a estrutura de diretórios da produção em /tmp/teste_generico e executa o rf_previsto.py real em cinco cenários (lista, intervalo, fallback do GFS, horizonte absoluto e falha controlada).

11. Segurança

O script semanal contém as credenciais do lftp (herdadas do shell script original) nas constantes LFTP_USUARIO e LFTP_SENHA. Recomenda-se movê-las para variáveis de ambiente ou para o arquivo ~/.netrc e restringir a permissão de leitura dos scripts.

12. Suporte

Modelo: Alberto Setzer (alberto.setzer@inpe.br) · Código NCL original: Guilherme Martins (guilherme.martins@inpe.br) · Programa Queimadas: <http://www.inpe.br/queimadas/>